



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2213 — Lógica para Ciencia de la Computación —
IIC2214 — Teoría de la Computación —

Pauta TAREA 3

1er semestre 2026

Publicación: Martes 21 de abril.

Entrega: **Lunes 04 de mayo hasta las 23:59 horas, via buzón en Canvas.**

Indicaciones

- La tarea es **individual**. Los casos de copia serán sancionados con la reprobación del curso con nota 1,1.
- Cada una de las dos preguntas tiene una nota entre 1 y 7 (hay un punto base). La nota de la tarea es el promedio de ambas preguntas.
- La tarea **debe** ser escrita en $\text{\LaTeX}$. No se aceptarán tareas escritas de otra forma. No olvide indicar su **nombre, número de alumno/a y sigla del curso (IIC2213 o IIC2214)**.
- No se aceptan atrasos (salvo que utilice su cupón `#problemaexcepcional`).

Pregunta 1

- (a) (3.0 pts) Sean $K, L \subseteq \Sigma^*$ dos lenguajes no vacíos. Demuestre que si K es finito, y $L \neq \Sigma^*$, entonces $K \leq_m L$.
- (b) (3.0 pts) Dé un ejemplo de dos lenguajes K y L tal que $K \not\leq_m L$.

Solución

(a) Como L es no vacío, existe una palabra $z_{yes} \in \Sigma^*$ tal que $z_{yes} \in L$. Como $L \neq \Sigma^*$, existe una palabra $z_{no} \notin L$. Considere la función $f : \Sigma^* \rightarrow \Sigma^*$ tal que $f(w) = z_{yes}$ si $w \in K$, y $f(w) = z_{no}$ si $w \notin K$. Por construcción, se cumple para todo $w \in \Sigma^*$ que:

$$w \in K \iff f(w) \in L.$$

Además f es computable. Una MT M para computar f es la siguiente: dada entrada $w \in \Sigma^*$, verificamos si $w \in K$. Si esto es así, retornamos z_{yes} . En caso contrario, retornamos z_{no} . Notar que la MT M puede almacenar de antemano todo el lenguaje K , ya que este es finito. Esto le permite hacer el chequeo $w \in K$. Además, la MT M tiene almacenada de antemano las palabras z_{yes} y z_{no} , para poder retornarlas cuando corresponda.

(b) Podemos considerar el lenguaje Accept , visto en clases:

$$\text{Accept} = \{\langle M, w \rangle \mid M \text{ es MT, } w \in \{0, 1\}^*, \text{ y } M \text{ acepta } w\}.$$

En clases se demostró que Accept es recursivamente enumerable y que $\overline{\text{Accept}}$ no es recursivamente enumerable. También se demostró la siguiente propiedad: si $K \leq_m L$ y L es recursivamente enumerable, entonces K es recursivamente enumerable. Luego, no podemos tener que $\overline{\text{Accept}} \leq_m \text{Accept}$, ya que esto implicaría que $\overline{\text{Accept}}$ es recursivamente enumerable. Concluimos que $\overline{\text{Accept}} \not\leq_m \text{Accept}$.

Pregunta 2

Un lenguaje $L \subseteq \Sigma^*$ es *cerrado bajo concatenación* si para todo $u, v \in \Sigma^*$ se cumple:

$$u \in L \text{ y } v \in L \implies uv \in L.$$

Definimos el siguiente lenguaje sobre $\{0, 1\}$:

$$\text{Concat} = \{\langle M \rangle \mid M \text{ es una MT y } L(M) \text{ es cerrado bajo concatenación}\}.$$

- (a) (3.0 pts) Demuestre que Concat no es recursivamente enumerable.
- (b) (3.0 pts) Demuestre que $\overline{\text{Concat}}$ no es recursivamente enumerable.

Solución

(a) Demostraremos que $\overline{\text{Accept}} \leq_m \text{Concat}$. Como en clases se vio que $\overline{\text{Accept}}$ no es recursivamente enumerable, obtenemos el resultado. Recordar que $\overline{\text{Accept}} \leq_m \text{Concat}$ es equivalente a demostrar que $\text{Accept} \leq_m \overline{\text{Concat}}$ (visto en clases). Considere la función f definida como:

$$f(\langle M, w \rangle) = \langle P_{M,w} \rangle,$$

donde la MT $P_{M,w}$ hace lo siguiente:

Sobre entrada $x \in \{0, 1\}^*$:

1. Si $x \neq 0$, **rechazar**.
2. Si $x = 0$, borrar x , escribir w en la cinta y ejecutar M con w . (retornar lo mismo que M .)

La función f es computable y se cumple lo siguiente:

$$\begin{array}{llll} M \text{ acepta } w & \implies & L(P_{M,w}) = \{0\} & \implies & \langle P_{M,w} \rangle \notin \text{Concat} \\ M \text{ no acepta } w & \implies & L(P_{M,w}) = \emptyset & \implies & \langle P_{M,w} \rangle \in \text{Concat} \end{array}$$

Notar que $\{0\}$ no es cerrado bajo concatenación, ya que $00 \notin \{0\}$. De hecho, el argumento sigue funcionando si reemplazamos la palabra 0 por cualquier otra palabra no vacía. Por otra parte, notar que el lenguaje vacío es cerrado bajo concatenación (por vacuidad).

Obtenemos que:

$$\langle M, w \rangle \in \text{Accept} \iff \langle P_{M,w} \rangle \in \overline{\text{Concat}}.$$

Luego f es reducción de Accept a $\overline{\text{Concat}}$.

(b) Demostraremos que $\overline{\text{Accept}} \leq_m \overline{\text{Concat}}$. Como en clases se vio que $\overline{\text{Accept}}$ no es recursivamente enumerable, obtenemos el resultado. Recordar que $\overline{\text{Accept}} \leq_m \overline{\text{Concat}}$ es equivalente a demostrar que $\text{Accept} \leq_m \text{Concat}$ (visto en clases). Considere la función f definida como:

$$f(\langle M, w \rangle) = \langle Q_{M,w} \rangle,$$

donde la MT $Q_{M,w}$ hace lo siguiente:

Sobre entrada $x \in \{0,1\}^*$:

1. Si $x \neq 01$, **aceptar**.
2. Si $x = 01$, borrar x , escribir w en la cinta y ejecutar M con w .
(retornar lo mismo que M .)

La función f es computable y se cumple lo siguiente:

$$\begin{array}{llll} M \text{ acepta } w & \implies & L(Q_{M,w}) = \{0,1\}^* & \implies & \langle Q_{M,w} \rangle \in \text{Concat} \\ M \text{ no acepta } w & \implies & L(Q_{M,w}) = \{0,1\}^* \setminus \{01\} & \implies & \langle Q_{M,w} \rangle \notin \text{Concat} \end{array}$$

Notar que $\{0,1\}^* \setminus \{01\}$ no es cerrado bajo concatenación, ya que $0 \in \{0,1\}^* \setminus \{01\}$ y $1 \in \{0,1\}^* \setminus \{01\}$, pero $01 \notin \{0,1\}^* \setminus \{01\}$. De hecho, el argumento sigue funcionando si reemplazamos la palabra 01 por cualquier otra palabra de largo 2 o más. Por otra parte, notar que el lenguaje $\{0,1\}^*$ es trivialmente cerrado bajo concatenación.

Obtenemos que:

$$\langle M, w \rangle \in \text{Accept} \iff \langle Q_{M,w} \rangle \in \text{Concat}.$$

Luego f es reducción de Accept a Concat .